CogLean in five slides

Why prove, what is proved, how, is it faithful, and where it goes

Jeff Shrager

October 2026

Three kinds of evidence a theory can give

One run	**Every run**	**Every model satisfying the postulates**
one model, one parameter setting	one model, all its executions	all models the theory allows
the theory *can* do it	that model *always* does it	the postulates *entail* it
simulation	model checking, sweeps	proof

CogLean is built around the third kind, and keeps the other two in the same language.

Confident errors

Let a familiar problem q have *sharp* associations (only its correct sum clears the criterion), and let x, with a different sum, be recognized as q. Then:

1. on q, whole-problem retrieval never errs;
2. on x, every whole-problem answer is wrong;
3. x uses the backup exactly as rarely as q does: $(1 - R_q)^n$.

In DOA alone, errors and backup use rise together. Confusion breaks the tie: a retrieved, wrong answer with no extra backup use.

Confident errors: the proof is a composition

```
theorem confident_errors (correct : P → A) (x q : P) (hT : ∀ y, total as (str y) ≠ 0)
    (hbx : mass (backup x) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieve (code q) = pure (some (code q)))
    (hx : retrieve (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    prob (recognizeRetrieve code retrieve str as cc n backup q) (wholeWrong correct q) = 0
    ↪ ∧
      prob (recognizeRetrieve code retrieve str as cc n backup x) (wholeWrong correct x) =
        prob (recognizeRetrieve code retrieve str as cc n backup x) isWhole ∧
      prob (recognizeRetrieve code retrieve str as cc n backup x) isBackup =
        prob (recognizeRetrieve code retrieve str as cc n backup q) isBackup := by
  refine ⟨confusion_gives_neighbours_answer code retrieve str as cc n backup correct q q
  ↪ (hT _) hq hs,
    wrong_eq_whole_of_sharp code retrieve str as cc n backup correct x _ _ (hT _) hx hs
    ↪ hne, ?_⟩
  rw [backup_prob_certain code retrieve str as cc n backup x _ hx hbx hT,
    backup_prob_certain code retrieve str as cc n backup q _ hq hbq hT]
```

Results: no failures

Check	Lean theorem	kind	tests	failures
Lean exact	`#eval MD.trial`	prob	243	0
P1	`columns_prob`	prob	394	0
P2	`whole_depends_only_on_route`	prob	8543	0
P3	`confusion_gives_neighbours_answer`	exact	200	0
P4	`confident_errors`	exact+prob	222	0
P7	`learning_preserves_trial`	exact	74692	0
P9	`columns_place_value`	exact	248800	0
P11	`discrimination_removes_confusion`	exact	91090	0
P6a, P6b, P14, P17, P18			0	*vacuous*

The prose and the formalization agree on recognition, EPAM learning, retrieval, both backups, and carrying.

The matrix

	Certainty	Stability	Order-free	Recency	Stored-only	Determinate
Discrimination	**has**	**has**	lacks	lacks	**has**	**has**
Activation	lacks	lacks	lacks	**has**	**has**	lacks
Strength	lacks	lacks	**has**	lacks	**has**	lacks
Global matching	lacks	lacks	**has**	lacks	lacks	**has**

- **has**: a general proof, for every parameter choice meeting stated conditions.
- lacks: a counterexample, computed exactly on the eight 3-bit codes.
- Every law is had by some family and lacked by another; every family has its own row. A script checks both.